

AI 기반 Multi-Agent SAST

for Raspberry Pi UserLand

대규모 C/C++ 코드를 배치로 나누고, 후보 → 문맥 → 검증 → 보고서로 연결

654 files

235,816 lines

6 agents

모든 결과는 취약점 확정이 아닌 근거 기반 검토 결과

오래된 대규모 저장소를 어떻게 빠르고 믿을 만하게 볼 것인가?

235,816

줄의 C/C++ 코드

583

초기 검토 후보

10

우선 분석 사례

단순 검색의 문제

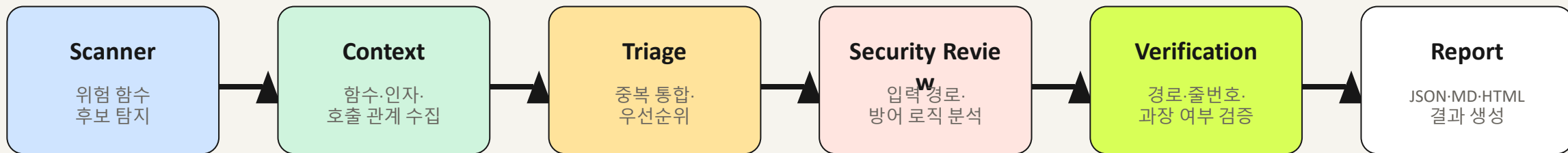
함수 이름이 있다는 사실만으로는 실제 취약점인지 알 수 없다.
중복·오탐·문맥 부족 때문에 사람이 모든 결과를 읽기 어렵다.

이 과제의 목표

대규모 코드를 안정적으로 나누고, 입력 경로와 안전장치를 근거로 확인한 뒤
'가능성 높음·낮음·정보 부족'으로 정직하게 구분한다.

핵심 원칙 위험 함수 ≠ 취약점 확정

역할을 분리해 한 단계의 실수를 다음 단계가 다시 확인



입력

target/userland + config/rules.json

출력

candidates → context bundles → analysis → reports

AI API 미설정

프롬프트·기대 JSON 스키마·실행 로그를 남기고, 기존 검토 데이터는 human_provided_review로 구분

세 배치로 균등하게 나누되 후보는 빠짐없이 유지

전체 후보와 일치 583 = 583

BATCH-001

78,749줄

209개 파일

후보 378개

0.756초 · 오류 0

BATCH-002

78,395줄

177개 파일

후보 101개

0.553초 · 오류 0

BATCH-003

78,672줄

268개 파일

후보 104개

0.554초 · 오류 0

코드 줄 수 기준으로 균형 분할

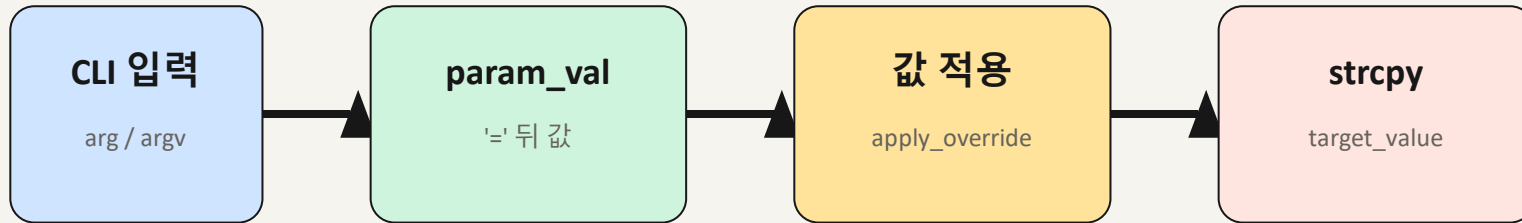
후보 분포는 코드 성격에 따라 달라짐

전체 코드를 보내지 않고 판단에 가까운 근거부터 수집

01	후보 우선순위화	위험도와 근거가 높은 후보부터 분석
02	함수 문맥 제한	후보 함수 140줄 · 묶음 최대 220줄
03	호출 관계 제한	호출자 2개 · 피호출자 3개 코드 우선
04	중복 제거	같은 원인의 후보를 한 분석 사례로 통합
05	필요할 때만 확장	부족한 함수·매크로만 추가 문맥 수집

AN-010 · CLI 입력이 256바이트 배열의 strcpy까지 도달

검토 결과: 취약 가능성 높음 · 신뢰도 97



dtoverlay.c:1852

```
static char target_value[256];
```

dtoverlay.c:1859

```
strcpy(target_value, override_value);
```

확인된 근거

dtoverlay_main.c:416–448에서 명령행 값 추출·전달
dtmerge.c:165–185에서도 argv 값이 같은 경로로 전달
복사 전 255자 제한 검사는 확인되지 않음

아직 확인하지 못한 것

동적 재현 · 실제 공격 성공 · 실행 권한·배포 조건
따라서 ‘취약점 확정’이 아니라 코드상 가능성으로 보고

위험 함수 탐지에서 멈추지 않고 근거와 반대 근거까지 기록

단순 grep

함수 이름 발견
→ 곧바로 위험 취급

이 워크플로

후보 → 문맥 → 중복 통합
→ 근거 분석 → 독립 검증 → 보고

신뢰성 장치

경로·줄번호 대조
정보 부족 명시
사람 검토 데이터 구분

62

검증된 인용

8

가능성 낮음

1

정보 부족

1

가능성 높음

human_provided_review

외부 LLM API 미사용 · 자동 AI 분석처럼 표시하지 않음

빠른 선별은 완료 — 다음은 실제 재현과 파서 정확도 개선

현재 결론

대규모 코드도 배치와 역할 분담으로
검토 가능한 근거 묶음으로 줄일 수 있다.

취약점 확정 아님

한계

정규식 기반 호출 관계
매크로·함수 포인터 해석 한계

다음 1

AN-010 최소 재현 테스트
ASan/UBSan 증거 수집

다음 2

AN-007의 libfdt 문자열 종료.
DTB 검증 경로 추가 조사

최종 메시지

찾는 속도는 AI로, 믿을 수 있는 결론은 근거와 검증으로